

EntraID / 3LO: AC Gateway + AC Identity

User-Delegated Auth for AI Agents

Deep Research · Architecture · Production Playbook
GENAIAF-562 · GENAI Agent Foundry

Epic	GENAIAF-562 — EntraID 3LO: AC Gateway + AC Identity
Scope	3-Legged OAuth enabling agents to act on behalf of users
Consumers	M365 (Graph API), GHE (GitHub Enterprise), Atlassian (Jira/Confluence)
Status	Blocked on dependencies: AC Gateway & AC Identity whitelisting
Compiled	June 2026

0. MASTER RESEARCH PROMPT (End-to-End Lifecycle)

The following prompt was used to drive this research across industry sources, standards bodies, cloud providers, and open-source communities. Use it to refresh or extend this document.

You are a senior identity architect at a large enterprise bank deploying AI agents that must act on behalf of authenticated users across M365 (Graph API), GitHub Enterprise (GHE), and Atlassian (Jira/Confluence). The identity platform is Microsoft Entra ID.

Research and document the COMPLETE end-to-end lifecycle of user-delegated 3-Legged OAuth (3LO) for AI agents, covering:

1. THEORY & STANDARDS — RFC 6749 Auth Code grant, RFC 8693 Token Exchange (OBO), OAuth 2.1, PKCE, IETF draft-oauth-ai-agents-on-behalf-of-user, MCP OAuth 2.1 spec.
2. MICROSOFT ENTRA SPECIFIC — Agent Identity Blueprint vs Agent Identity, Federated Identity Credentials (FIC), Managed Identity as credential, OBO flow token chain (T1→Tc→resource token), Microsoft.Identity.Web + AgentCore SDK, Conditional Access for agents.

3. CLOUD-PROVIDER IMPLEMENTATIONS — AWS Bedrock AgentCore Identity (3LO, secure token vault, KMS), sidecar/auth-proxy pattern, Workload Identity Federation.
4. API GATEWAY PATTERNS — AC Gateway as policy enforcement point, inbound/outbound auth planes, per-user credential vault, token refresh orchestration, MCP Gateway, Open Policy Agent (OPA).
5. TARGET RESOURCE INTEGRATION — Atlassian 3LO (Jira/Confluence scopes, rotating refresh tokens, cloudId), GHE OAuth App flow, M365 Graph delegated permissions, CIBA for high-risk actions.
6. SECURITY & ZERO TRUST — CSRF/PKCE binding, least-privilege scopes, token isolation per user/tenant, refresh-token rotation, revocation, audit trail (user + agent + scope + timestamp).
7. PRODUCTION HARDENING — Proactive refresh at 80% TTL, distributed locking, secret management (no client secrets), FIC + MSI preferred, monitoring/alerting, OWASP Agentic Top 10.
8. DEPENDENCIES & BLOCKERS — AC Gateway whitelisting for AgentCore, AC Identity whitelisting (Stage 1), FT/Entra self-service engagement with Security.
9. MINIMAL DELIVERY CHECKLIST — EntraID 3LO pattern documentation (≥1 scope), Security/Identity team alignment on consent flow, reference implementation for user-delegated token flows.
- For each area: describe the industry best practice, name who has productionised it, and note gaps or open questions.

1. BACKGROUND & PROBLEM STATEMENT

1.1 Why User-Delegated Auth Matters for AI Agents

Traditional OAuth assumes an **interactive human session**: a redirect, a consent screen, and a short-lived browser session. AI agents invert this model — they must act **autonomously, after the user session ends**, on behalf of users who explicitly consented. This creates a fundamentally different lifecycle requirement.

A simple 'redirect and store the token' approach works in demos but fails under four production conditions: (1) tokens expire while the agent is mid-task, (2) background refresh races cause 401 cascades, (3) multi-user/multi-tenant systems cannot share tokens, and (4) enterprise security reviews reject service accounts with broad static credentials.

1.2 The Three Identity Questions Agents Must Answer

Question	Traditional answer	Agent-specific answer
Who is acting?	The authenticated user	The agent acting FOR the user (dual identity)
What can they do?	User's role/permission set	Intersection of user permissions AND agent-granted scopes
For how long?	Browser session lifetime	Until task complete or token revoked — even days later

1.3 Epic Context (GENAIAF-562)

Consumer view	Access M365, GHE, Atlassian with specific (not global) permissions
---------------	--

Builder view	Agent acts on behalf of user (3LO) for Graph API; design agentic apps with modern AuthNZ
Security view	Agent Identity and Permission to be a solved, auditable topic
Scope	3-Legged OAuth enabling agents to act on behalf of users
Blocked on	AC Gateway whitelisting (GENAITB-201) + AC Identity whitelisting (Stage 1) + FT/Entra self-service

2. STANDARDS & THEORETICAL FOUNDATION

2.1 Core RFCs

RFC / Spec	What it defines	Agent relevance
RFC 6749	OAuth 2.0 — Auth Code Grant (3LO), Client Credentials (2LO)	Foundation for all agent auth flows
RFC 7636 PKCE	Proof Key for Code Exchange — binds code to verifier	Mandatory for public clients; prevents code injection
RFC 8693	Token Exchange — OBO flow: swap user token for agent-scoped token	Core mechanism for agent acting on behalf of user
RFC 8707	Resource Indicators — bind token to specific resource server	Prevents token audience confusion in multi-API environments
OAuth 2.1	Consolidates 2.0 + BCP; mandates PKCE; removes implicit/password grant	MCP spec mandates OAuth 2.1 for all HTTP transports
IETF Draft OBO-AI	Extends Auth Code flow: requested_actor + actor_token params	Emerging standard; agent named on consent screen
MCP 2025-11-05	Model Context Protocol mandates OAuth 2.1 + PKCE for remote servers	All MCP-based tool calls require compliant auth

2.2 The 3LO Authorization Code Flow — Step by Step

The 3-Legged OAuth flow involves three parties: the **Client** (the AI agent/app), the **Authorization Server** (Entra ID / Atlassian auth), and the **Resource Owner** (the user). Unlike 2LO (machine-to-machine), 3LO requires explicit user consent before any token is issued.

Step	Actor	Action / Token
1	Agent/App	Redirect user to /authorize?client_id=...&scope;=...&code_challenge=PKCE_HASH&state;=CSRF_TOKEN
2	Auth Server	Display consent screen showing what the agent will access on user's behalf
3	User	Grant (or deny) access; Auth Server issues short-lived authorization_code
4	Agent/App	POST to /token: exchange code + code_verifier for access_token + refresh_token
5	Agent	Call resource API (Graph, Jira, GHE) with Bearer access_token
6	Agent (background)	Proactively refresh at 80% TTL; rotate refresh_token; store securely
7	On revoke/task end	Invalidate refresh_token; remove from credential vault

2.3 On-Behalf-Of (OBO) — Token Exchange for Agent Chains

RFC 8693 Token Exchange is the mechanism where an agent receives a *user token* (*Tc*) from the calling client application, then exchanges it for a *resource-scoped token* at the target API. This creates a clear delegation chain without embedding user credentials in the agent.

```
POST /oauth2/v2.0/token grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer
assertion={user_access_token_Tc} client_assertion={agent_identity_token_T1} # FIC/MSI preferred
requested_token_use=on_behalf_of scope=https://graph.microsoft.com/Mail.Read
```

Key constraint: the output token is bounded by the **intersection** of (user permissions) AND (agent-granted scopes). The agent cannot escalate beyond what the user already has.

3. MICROSOFT ENTRA ID — AGENT IDENTITY ARCHITECTURE

3.1 The Three-Tier Agent Identity Model

Microsoft Entra Agent ID (GA 2025/2026) introduces a purpose-built identity model for agents that separates **identity definition** from **identity instances**:

Tier	Object	Purpose
1 — Blueprint	Agent Identity Blueprint (App Registration)	Template defining agent type, permissions, credential type. Admin sets Conditional Access here. One blueprint → many instances.
2 — Identity	Agent Identity (Service Principal child)	Per-deployment instance. Holds scoped permissions. Can impersonate Agent User. Token exchange uses FIC → MSI chain.
3 — User	Agent User Account (optional)	Synthetic user principal for agents that need a mailbox, calendar, or full user-scoped Graph access.

3.2 Token Chain for Interactive (OBO) Agent Flow

The token chain follows a strict three-step sequence. Each step produces a token that serves as the credential for the next step:

```
Step 1: Blueprint acquires T1 POST /oauth2/v2.0/token client_id=AgentBlueprint
grant_type=client_credentials client_assertion=TUAMI # Managed Identity token
scope=api://AzureADTokenExchange/.default Step 2: Agent Identity performs OBO exchange using T1 +
Tc POST /oauth2/v2.0/token client_id=AgentIdentity
grant_type=urn:ietf:params:oauth:grant-type:jwt-bearer assertion={Tc} # User token received from
client app client_assertion={T1} # Blueprint token requested_token_use=on_behalf_of
scope=https://graph.microsoft.com/Mail.Read Step 3: Agent calls resource API with resulting
delegated token
```

■ Client secrets **MUST NOT** be used in production. Use **Federated Identity Credentials (FIC)** with **Managed Identity (MSI)** as the credential for **Agent Identity Blueprints**.

3.3 Supported Grant Types for Agent Entities

Grant Type	Supported by	Use case
client_credentials	Blueprint + Identity	Bootstrap token for FIC chain; autonomous app-only operations
jwt-bearer (OBO)	Blueprint + Identity	User-delegated access; agent acts on behalf of signed-in user
refresh_token	Agent Identity	Background long-running tasks; maintain user context across sessions
user_fic	Agent Identity	Agent's user account impersonation (mailbox, calendar scenarios)

■ Interactive /authorize flows are **EXPLICITLY BLOCKED** for agent entities. Redirect URIs are not supported. All auth occurs programmatically via token exchange.

3.4 Microsoft Entra Agent ID — Governance Capabilities

Capability	Mechanism	Benefit
Conditional Access	Apply CA policies to Blueprint → all instances inherit	Enforce MFA, location, risk-based controls on agents
Access Packages	Entitlement Management API assigns scoped permissions	Governed, time-bound access; auto-expiry with sponsor notification

Capability	Mechanism	Benefit
Access Reviews	Periodic review of agent identity permissions	Compliance: verify agent still needs its access
Lifecycle Workflows	Automate sponsor handoff, expiry notifications	Human accountability chain for every agent
Identity Protection	Risk scoring, anomaly detection on agent tokens	Detect token abuse or compromised agent instances
Audit Logs	Full audit trail: agent ID + user OID + scope + timestamp	Forensics, regulatory compliance

4. AC GATEWAY — THREE-PLANE ARCHITECTURE

4.1 Why a Gateway Is Not Just a Proxy

Traditional API gateways sit *in front of* APIs, controlling inbound traffic. Agentic architectures are different — the agent itself mediates between user and infrastructure. The **AC Gateway** must therefore operate as a **policy enforcement point at the execution layer**, not just as an edge router. It owns three distinct planes:

Plane	Responsibility	Key mechanism
Inbound Auth	Validate who is calling the agent	Verify Entra bearer token; extract user OID + agent identity
Access Control	What can this agent do for this user?	OPA/PDP policy: RBAC + ABAC + approval thresholds
Outbound Auth	Credential to call downstream API	Per-user credential vault; 3LO refresh token → access token; agent never sees raw credentials

4.2 Token Lifecycle at the Gateway

The gateway is responsible for the full token lifecycle for 3LO integrations. It must manage per-user, per-provider credential storage and proactive refresh:

Event	Gateway action
User first consent	Store {access_token, expires_at, refresh_token, scopes, provider_metadata} encrypted, keyed by (user_id, provider)
Agent makes tool call	Lookup credential vault; if access_token valid → inject; if expiring → refresh first
Token near expiry (80%)	Background proactive refresh; acquire per-(user,provider) distributed lock to avoid race conditions
Refresh + rotate atomically	New access_token + new refresh_token written; old refresh_token invalidated immediately
User revokes access	Delete vault entry; agent blocked from further calls for that user+provider
Agent task completes	Optionally revoke refresh_token; agent should not retain delegation between unrelated tasks

4.3 2LO vs 3LO Decision Framework (per target API)

Condition	Use 2LO (Client Credentials)	Use 3LO (Auth Code)
Data ownership	Org-level / shared data (no user owner)	User's personal data (email, calendar, issues assigned to them)
Consent	Admin pre-consents once for all agents	Each user must explicitly consent (GDPR requirement)
Token subject	Application identity (agent)	User identity (delegated; user + agent named)
Examples	Internal analytics API, batch processors	Graph Mail, Jira issues, GHE repo access
AC Gateway config	Store app credential (FIC/cert)	Store per-user refresh_token; run consent redirect flow

4.4 Production Runtime Flow (End-to-End)

The sequence for a single 3LO agent tool call via AC Gateway:

#	Stage	Detail
1	User prompt	User: 'Close ticket DEMO-123 in Jira'
2	Agent plan	LLM determines: call Jira API → update issue status
3	Gateway inbound	Validate user Entra bearer token; extract user_oid
4	PDP check	RBAC: does agent have Jira:write scope for this user? ABAC: within policy? Approval needed?
5	Vault lookup	Retrieve Jira refresh_token for user_oid; check if access_token valid
6	Token refresh	(If needed) POST to auth.atlassian.com/oauth/token with refresh_token; store rotated pair
7	Tool execution	Gateway (not agent) executes PATCH api.atlassian.com/ex/jira/{cloudId}/rest/api/3/issue/DEMO-123
8	Audit log	Record: agent_id, user_oid, scope=write:jira-work, resource=DEMO-123, result, timestamp
9	Response	Return result to agent; agent formats response to user

5. TARGET RESOURCE INTEGRATIONS

5.1 Microsoft 365 — Graph API (Delegated Permissions)

Graph API is the primary target for M365 user-delegated access. The agent must use the OBO flow (Section 3.2) to exchange the user's Entra token for a Graph-scoped delegated token.

Scope	Access granted	Notes
Mail.Read	Read user's email	Delegated; requires user consent
Mail.Send	Send email as user	High risk; consider CIBA approval gate
Calendars.ReadWrite	Read/write user calendar	Delegated
Files.ReadWrite	Read/write OneDrive files	Delegated
User.Read	Read signed-in user profile	Always needed; low risk
.default	All pre-consented admin scopes	AVOID for 3LO — use minimal explicit scopes

✓ Best practice: request only the scope needed for the current task. Use incremental consent to add scopes as needed.

5.2 GitHub Enterprise (GHE) — OAuth App / GitHub App

GHE supports both OAuth Apps (user-delegated 3LO) and GitHub Apps (installation-scoped 2LO). For agent access to user-owned repositories, OAuth App with 3LO is required. The AC Gateway must be whitelisted as the redirect URI in the GitHub OAuth App config.

Aspect	Detail
Authorization URL	<code>https://{ghe-host}/login/oauth/authorize?client_id=...&scope;=repo read:user</code>
Token URL	<code>https://{ghe-host}/login/oauth/access_token</code>
Scopes	repo (full repo access), read:org, read:user, workflow — use minimal set
Token type	GitHub does not issue refresh tokens for OAuth Apps; access tokens are long-lived (use with caution)
GitHub App alt.	Installation tokens (2LO) expire after 1hr; fine-grained permissions; recommended for server-to-server
Gateway config	Whitelist AgentCore/AC Gateway callback URL in GHE OAuth App settings

5.3 Atlassian (Jira / Confluence) — 3LO Details

Atlassian's 3LO uses Auth Code Grant with rotating refresh tokens. All API calls must go through `api.atlassian.com` (not the tenant domain) and include the `cloudId`.

```
Authorization URL: https://auth.atlassian.com/authorize ?client_id=YOUR_CLIENT_ID
&scope;=read:jira-work write:jira-work read:jira-user offline_access
&redirect;_uri=https://ac-gateway.bank.internal/callback/atlassian &state;={CSRF_TOKEN}
&response;_type=code &prompt;=consent Token Exchange: POST https://auth.atlassian.com/oauth/token
{ grant_type: authorization_code, client_id, client_secret, code, redirect_uri } API Call: GET
https://api.atlassian.com/ex/jira/{cloudId}/rest/api/3/issue/DEMO-123 Authorization: Bearer
{access_token}
```

Key scope	Permission granted
read:jira-work	Read issues, projects, workflows
write:jira-work	Create/update issues, transitions

Key scope	Permission granted
read:jira-user	Read user profiles
read:confluence-content.all	Read Confluence pages and spaces
write:confluence-content	Create/update Confluence content
offline_access	REQUIRED to receive refresh_token for background agent operation
manage:jira-configuration	Admin config — AVOID unless explicitly needed

- ✓ Atlassian: user permissions always constrain the app. If user lacks Administer Jira, the agent cannot administer Jira even if scope is granted. Least privilege is automatic.
- ✓ Rotating refresh tokens: each use issues a new refresh_token. Gateway must store the latest one atomically.

6. INDUSTRY: WHO HAS PRODUCTIONISED 3LO FOR AI AGENTS

Organisation	Product / Pattern	Key production details	Status
AWS / Amazon Bedrock	AgentCore Identity	Full 3LO + 2LO. Secure token vault with KMS encryption. Per user+agent credential isolation. @requires_access_token decorator handles entire consent + refresh flow. Pre-built integrations: GitHub, Slack, Salesforce, Google Drive.	GA (no-cost preview ended Sept 2025)
Microsoft	Entra Agent ID + Microsoft.Identity.Web	Blueprint→Identity→OBO chain. FIC+MSI as preferred credential (no secrets). services.AddAgentIdentities() in .NET SDK. Agent 365 admin center for governance. Conditional Access, Identity Protection, Access Reviews all extended to agents.	GA 2026 (beta APIs → GA)
Atlassian	3LO for Forge/Connect apps	Rotating refresh tokens. offline_access scope for background agents. cloudId-based API routing. Account-level consent (per-user, per-site). Atlassian Government Cloud variant (AGC developer console).	Production (long-standing)
TrueFoundry	MCP Gateway + Token Vault	Three-plane architecture (inbound/access/outbound). Per-(user,provider) distributed lock for refresh. Proactive refresh at 80% TTL. Atomic rotation of access+refresh tokens. 3LO for GitHub/Slack; 2LO for internal services.	Production (documented May 2026)
WorkOS	Auth platform for AI agents	OAuth On-Behalf-Of flow (IETF draft). Front-channel consent with named agent on screen. act claim in resulting token names user + client + agent. Production MCP pilot programs noted to stall without scoped permissions and audit trail.	Production 2025-2026
Aembit / SecureW2	Workload Identity Federation	Eliminate static secrets entirely. Agent proves identity via SPIFFE/SPIRE workload identity. Conditional Access at network layer. Zero standing privileges.	Production (enterprise customers)
Auth0 / Okta	AI Agent auth with CIBA	CIBA (Client-Initiated Backchannel Authentication) for high-risk agent actions. Push notification to user device for explicit real-time consent before critical operations (e.g., transfer funds, send company-wide email). FGA for fine-grained tool authorization.	Production 2025
Microsoft (n8n example)	n8n + Entra Agent ID	Community node n8n-nodes-entraagentid. OBO webhook flow. Auth Manager acquires delegated token for Agent User via Blueprint FIC credentials. AES-256-GCM token caching within workflow run. MCP Graph Server for Enterprise.	Reference impl (June 2026)

6.1 Key Production Patterns Observed

- No client secrets in production — universally replaced by FIC + Managed Identity or certificates.
- Sidecar/auth-proxy pattern — agent never handles credentials; a companion container/service mediates all token operations.
- Proactive refresh at 80% TTL with distributed locking — prevents 401 cascades in concurrent agent tasks.
- Atomic token rotation — new access_token + refresh_token written in single transaction; old refresh immediately invalidated.
- Per-user credential isolation — tokens stored with (user_id, provider) key; zero cross-user sharing enforced at storage layer.
- Revoke on task completion — refresh tokens should not persist between unrelated agent tasks.
- Full audit trail — every API call tagged: agent_id, user_oid, scope, resource, timestamp, result.

7. SECURITY, ZERO TRUST & THREAT MODEL

7.1 PKCE & CSRF Binding (Mandatory for all 3LO flows)

PKCE (RFC 7636) prevents authorization code injection by binding the auth request to a verifier that only the original client can produce. The state parameter prevents CSRF. Both are mandatory in OAuth 2.1.

```
# Client generates: code_verifier = crypto.random(32) # 32-byte random
code_challenge = BASE64URL(SHA256(code_verifier)) # In /authorize request:
&code_challenge={code_challenge}&code_challenge_method=S256&state={session_bound_nonce} # In
/token request: &code_verifier={code_verifier} # Auth server verifies SHA256(verifier)==challenge
```

7.2 Threat Model & Mitigations

Threat	Attack vector	Mitigation
Token theft	Stolen access_token used by attacker	Short TTL (minutes). mTLS binding. Monitor for concurrent use from different IPs.
Refresh token theft	Stolen refresh_token used to mint new access tokens	Refresh token rotation: stolen token triggers detection (legitimate client fails next refresh).
Prompt injection → token misuse	Malicious input tricks agent into using tokens for unintended actions	PDP policy check at gateway; action is independent of LLM output. Zero trust: LLM not trusted for auth decisions.
Service account abuse	Agent holds global static credential; any injection = full access	3LO with per-user scoped tokens. Never use service accounts for user-data agents.
Token audience confusion	Token for gateway reused against backend APIs	RFC 8707 Resource Indicators: bind token to specific resource URI. MCP spec requires this.
Excessive scope creep	Agent accumulates more access than needed over time	Access Reviews (Entra). Time-bound Access Packages. Principle: request only scope needed for current task.
Dynamic Client Reg abuse	Malicious agent self-registers with broad scopes	Validate client metadata and redirect URIs strictly. Require pre-approval for new agent registrations.
Compromised agent instance	One agent instance with valid token attacks others	FIC + MSI credential bound to specific agent identity. No shared secrets between instances. mTLS/SPIFFE.

7.3 CIBA — Human-in-the-Loop for High-Risk Actions

Client-Initiated Backchannel Authentication (CIBA) is an OAuth 2.0 extension for decoupled, asynchronous user approval. It is the right mechanism when an agent wants to perform an **irreversible or high-risk action** and real-time user approval is required.

Step	Action
1	Agent identifies high-risk action (e.g., 'send email to all-company list', 'delete Jira project')
2	Agent sends CIBA authorization request to Auth Server
3	Auth Server sends push notification to user's mobile device
4	User approves or denies on device
5	Agent polls token endpoint; receives approval token or rejection
6	Agent proceeds (or aborts) based on user decision

■ Actions like sending bulk emails, deleting resources, or financial transfers should ALWAYS route through CIBA.
Design the agent's tool catalog with explicit risk tiers.

8. DEPENDENCIES, BLOCKERS & DELIVERY PLAN

8.1 Current Blockers (as of GENAIAF-562)

Dependency	Status	Ticket / Tracking	Resolution path
AC Gateway whitelisting for AgentCore service	Not yet available	GENAITB-201 (NEW) + GHE infra: Whitelist AgentCore Gateway for Stage 1	Engage Platform/Infra team; tie to Stage 1 milestone. Monitor GENAITB-201.
AC Identity whitelisting for AgentCore service	Not yet available	GENAITB-201 (NEW) + GHE infra: Whitelist AgentCore Identity for Stage 1	Same as above — co-track both whitelisting items together.
FT/Entra self-service registration	Currently manual — requires Security engagement	Not tracked separately (manual process)	Schedule Security/Identity team alignment meeting. Define consent flow approval process.

8.2 Minimal Delivery Checklist (from Epic)

#	Deliverable	Owner	Acceptance criteria
M D-1	EntraID 3LO pattern documented for ≥1 scope	Builder / Architect	Living document published in Confluence covering flow, token chain, scope selection, and gateway config for at least one of: M365 / GHE / Atlassian.
M D-2	Security/Identity team alignment on consent flow	Security + Product	Agreed consent UX pattern; defined which agent types require user consent vs admin pre-consent; CIBA policy for high-risk actions.
M D-3	Reference implementation for user-delegated token flows	Engineering	Working code (e.g., using Microsoft.Identity.Web + AddAgentIdentities()) demonstrating OBO chain: Blueprint → Identity → Resource API with FIC/MSI credentials.

8.3 Recommended Phased Delivery Roadmap

Phase	Milestone	Key activities
Phase 0 (Now — unblocked)	Alignment & Design	1. Document 3LO pattern (this research). 2. Security/Identity meeting: agree consent flow model. 3. Register OAuth apps in Atlassian & GHE dev consoles (can be done before gateway is live). 4. Define agent identity blueprint in Entra dev tenant.
Phase 1 (After Stage 1 whitelist)	AC Gateway + AC Identity online	1. GENAITB-201 unblocked: AC Gateway can route to AgentCore service. 2. AC Identity whitelisted: Entra Agent Identity can acquire tokens through gateway. 3. Implement credential vault (per-user, per-provider, AES-256 encrypted). 4. Implement proactive refresh with distributed locking.
Phase 2 (Reference impl)	End-to-End Working Flow	1. Reference impl: agent calls Jira/GHE/Graph via OBO chain. 2. Consent flow: user redirect → approval → refresh_token stored in vault. 3. Audit logging: agent_id + user_oid + scope + resource + timestamp. 4. Access Reviews configured in Entra for agent identities.
Phase 3 (Hardening)	Production Readiness	1. No client secrets anywhere: FIC + MSI for all blueprints. 2. Conditional Access policies on all agent blueprints. 3. CIBA gate for high-risk tool actions. 4. OWASP Agentic Top 10 review. 5. Runbook: revoke agent access, rotate credentials, incident response.

9. STRUCTURED DOCUMENT PLAN (What the Full Spec Should Cover)

This section defines the table of contents and depth requirements for the full EntraID 3LO Agent Auth specification document to be authored in Confluence.

Section	Title	Depth required	Primary audience
1	Problem Statement & Scope	1 page — why 3LO, what's in/out, epic context	All
2	Identity Architecture	Detailed: Blueprint/Identity/User tiers, FIC+MSI credential chain, Entra Agent ID objects	Architects, Security
3	Token Flow Diagrams	Sequence diagrams: OBO chain, 3LO consent flow, refresh flow, revocation	Engineers
4	AC Gateway Design	3-plane architecture, credential vault schema, refresh orchestration, OPA policy examples	Platform Engineers
5	M365 Graph Integration	Delegated permissions reference, OBO code example (.NET + Python), scope minimisation guide	Engineers
6	GHE Integration	OAuth App config, callback URL whitelisting, token TTL handling (no refresh), GitHub App alternative	Engineers
7	Atlassian Integration	3LO app setup, scope reference table, cloudId discovery, rotating refresh token handling, offline_access	Engineers
8	Consent Flow UX	User consent screen design, admin pre-consent scenarios, incremental consent, CIBA high-risk gate	Product, UX, Security
9	Security Controls	PKCE, CSRF, threat model, token isolation, audit requirements, OWASP Agentic Top 10 mapping	Security
10	Zero Trust Policy	Conditional Access policies, least-privilege scope design, Access Reviews, lifecycle workflows	Security, Governance
11	Reference Implementation	Working code: .NET Microsoft.Identity.Web + AddAgentIdentities(); Python MSAL; test harness	Engineers
12	Operational Runbook	Deploy new agent identity; rotate FIC credential; revoke user access; incident response; monitoring alerts	Ops, SRE
13	Dependency Tracker	GENAIB-201 status, GHE infra whitelist, FT/Entra self-service process, stage gates	PM, Delivery
14	Glossary	3LO, 2LO, OBO, FIC, MSI, PKCE, CIBA, SPIFFE, MCP, blueprint, agent identity	All

10. OPEN QUESTIONS & INDUSTRY GAPS

#	Open question / gap	Current state	Recommended action
O Q-1	IETF draft-oauth-ai-agents-on-b ehalf-of: timing for adoption?	Draft -02 (Aug 2025). Not yet adopted by OAuth WG. Core concepts (requested_actor, actor_token) likely to persist.	Track IETF OAuth WG. Design consent screen to show agent name now (can retrofit draft params later).
O Q-2	GitHub Enterprise: no refresh token for OAuth Apps — how to handle long-running agent tasks?	GHE OAuth App access tokens are long-lived (no expiry by default) — security risk.	Prefer GitHub App (installation token, 1hr TTL, fine-grained) for server-side agents. Use OAuth App only for user-interactive flows.
O Q-3	Atlassian consent: user must repeat per-site. How to handle multi-site agents?	Current Atlassian 3LO is per-site. Admin cannot revoke individual user grants; must uninstall app.	Document limitation. Design agent to request per-site consent lazily (just-in-time when user first uses that site).
O Q-4	CIBA: which tools in the agent catalog are 'high risk'?	Not yet defined for GENAI Agent Foundry agents.	Security/Product alignment session (MD-2 prerequisite). Define risk tiers: read / write / destructive / financial.
O Q-5	Multi-agent pipelines: how does trust chain propagate across agent hops?	RFC 8693 token exchange reduces authority at each hop (never amplifies). But audit chain across agents is not standardised.	Implement correlation_id + delegation_chain claim in internal tokens. Log each hop.
O Q-6	FT/Entra self-service: what is the approval process for registering new agent OAuth apps?	Currently manual, requires Security engagement.	Define and publish SLA: Security reviews new app registration within N days. Create request template.

EXECUTIVE SUMMARY

The EntraID 3LO pattern for AI agents is **well-understood, standardised, and production-proven** by AWS Bedrock AgentCore, Microsoft Entra Agent ID, Atlassian, and multiple enterprise platform teams. The core pattern is: **Agent Identity Blueprint** → **FIC/MSI credential** → **OBO token exchange** → **per-user scoped access token** → **resource API call** → **audit log**.

The main blocker for GENAIAF-562 is infrastructure whitelisting (AC Gateway + AC Identity), not a standards or design gap. The design work (this document, consent flow alignment, reference implementation) can proceed in parallel while GENAIB-201 is resolved.

The three non-negotiable production requirements are: **(1) no client secrets** — use FIC+MSI; **(2) per-user token isolation** — never share tokens across users; **(3) full audit trail** — every agent API call must be traceable to a specific agent, user, scope, and timestamp.